Lecture: 3
Stacks (Balanced Parenthesis & Infix, Postfix Notation)

The best data structure to check whether an arithmetic expression has balanced parenthesis is stack.

The way to write arithmetic expression is known as a notation.

Infix notation: The expression of the form **a op b** i.e. when an operator is in between every pair of operands.
Postfix (Reverse-Polish) notation: The expression of the form **a b op** i.e. when an operator is followed for every pair of operands.
Prefix (Polish) Notation: The expression of the form **op a b** i.e. when operator is written ahead of operands.

Sr.No.	Infix Notation	Prefix Notation	Postfix Notation
1	a + b	+ a b	a b +
2	(a + b) * c	* + a b c	a b + c *
3	a * (b + c)	* a + b c	a b c + *
4	a / b + c / d	+ / a b / c d	a b / c d / +
5	(a + b) * (c + d)	* + a b + c d	a b + c d + *
6	((a + b) * c) - d	- * + a b c d	a b + c * d -

The computer usually evaluates an arithmetic expression written in infix notation in two steps:

1. It converts the infix notation to postfix notation &
2. then it evaluates the postfix expression.

Why postfix representation of the expression?

The compiler scans the expression either from left to right or from right to left.
Consider the expression: **a+b*c+d**

- The compiler first scans the expression to evaluate the expression **b * c**,
- then again scans the expression to add **a** to it.
- the result is then added to **d** after another scan.

The repeated scanning makes it very in-efficient. It is better to convert the expression to postfix form before evaluation.

The corresponding expression in postfix form is: **abc*+d+**
The postfix expressions can be evaluated easily using a stack.

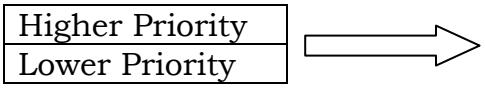
Precedence and Associativity of Operators:

- Operator precedence determines which operator is performed first in an expression with more than one operators with difference precedence.
- Operator associativity is used when two operators of same precedence appear in an expression. Associativity can be either Left to Right or Right to Left.
    - e.g. ***** and **/** have same precedence and their associativity is Left to Right, so the expression **100/10*10** is treated as **(100/10)*10.**

Operators	**Precedence**	**Associativity**
^ (exponentiation)	6	Right to left
*(Multiply), / (Division)	5	Left to Right
+(add), - (Subtract)	4	Left to Right
<,<=,<>,>=	3	Left to Right
AND	2	Left to Right
OR, XOR	1	Left to Right

Infix to Postfix Conversion:

In stack:



Nothing will Pop out.

but if stack contains a higher priority operation and then comes a lower priority operation → first higher priority operation will pop out.

In case of same precedence, No two operators of same priority can stay together in stack e.g. if / is there in top of stack and next we have is *, then first pop the / and then push the * in the stack.

Example:

Infix Expression Q: A + (B * C – (D / E ^ F) * G) * H

A	+	(	B	*	C	-	(	D	/	E	^	F	)	*	G	)	*	H	)
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

Symbol	**Scanned**	**Stack**	**Expression (P)**	**Remarks**
1	A	(	A	
2	+	(+	A	
3	(	(+(	A	
4	B	(+(	AB	
5	*	(+(*	AB	
6	C	(+(*	ABC	
7	-	(+(-	ABC*	multiply has higher priority than minus → first * will pop out.
8	(	(+(-(	ABC*	
9	D	(+(-(	ABC*D	
10	/	(+(-(/	ABC*D	
11	E	(+(-(/	ABC*DE	
12	^	(+(-(/^	ABC*DE	nothing got popped out because ^ has higher priority than /
13	F	(+(-(/^	ABC*DEF	
14	)	(+(-	ABC*DEF^/	whenever parenthesis will close all the operations will pop out
15	*	(+(-*	ABC*DEF^/	
16	G	(+(-*	ABC*DEF^/G	
17	)	(+	ABC*DEF^/G*-	
18	*	(+*	ABC*DEF^/G*-	
19	H	(+*	ABC*DEF^/G*-H	
20	)		ABC*DEF^/G*-H*+	

So, Postfix expression is**: ABC* DEF ^/G* - H*+**

Postfix to Infix Notation:

e.g. P: 5, 6, 2, +, *, 12, 4, /, -

	Symbol Scanned	**Stack**
(1)	5	5
(2)	6	5,6
(3)	2	5,6,2
(4)	+	5,8
(5)	*	40
(6)	12	40,12
(7)	4	40,12,4
(8)	/	40,3
(9)	-	37
(10)	)	

➔ **Q: 5 * (6+2) -12/4**